



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3101

Course Title: Operating System and System Programming Lab

Lab Report No: 07

Lab Title: Implementation of Round Robin CPU Scheduling Algorithm

Submitted To:

Monishanker Halder,
Assistant Professor,
Dept. of CSE, JUST

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 21th July, 2025

Experiment Title: Implementation of Round Robin CPU Scheduling Algorithm

1. Objective

The primary objective of this laboratory experiment is to implement the Round Robin (RR) CPU scheduling algorithm, one of the most commonly used preemptive scheduling algorithms in operating systems. The goal is to understand how time quantum affects process scheduling, calculate waiting times and turnaround times, and analyze the algorithm's performance characteristics.

2. Theory

2.1 CPU Scheduling

CPU scheduling is the process of determining which process should be executed by the CPU at any given time. The main objectives of CPU scheduling include:

- Maximizing CPU utilization
- Minimizing response time
- Ensuring fairness among processes
- Maximizing throughput

2.2 Round Robin Scheduling Algorithm

Round Robin is a preemptive scheduling algorithm that assigns a fixed time slice (quantum) to each process in a cyclic manner. Key characteristics include:

- **Preemptive:** Processes can be interrupted and resumed
- **Time Quantum:** Fixed time slice allocated to each process
- **Circular Queue:** Processes are organized in a circular ready queue
- **Fair Scheduling:** Each process gets equal CPU time allocation

2.3 Key Terminology

- **Burst Time:** Total CPU time required by a process
- **Waiting Time:** Time a process spends waiting in the ready queue
- **Turnaround Time:** Total time from process arrival to completion
- **Response Time:** Time from process arrival to first CPU allocation
- **Time Quantum:** Fixed time slice allocated to each process

3. Implementation

3.1 Source Code:

```
#include <bits/stdc++.h>
using namespace std;
```

```

struct Process
{
    string name;
    int burst_time;
    int remaining_time;
    int waiting_time;
    int turnaround_time;
};

void roundRobinScheduling(vector<Process> &processes, int time_quantum)
{
    queue<Process *> ready_queue;
    int current_time = 0;
    int completed = 0;
    int n = processes.size();

    for (auto &p : processes)
    {
        p.remaining_time = p.burst_time;
        ready_queue.push(&p);
    }

    while (!ready_queue.empty())
    {
        Process *current = ready_queue.front();
        ready_queue.pop();

        int execution_time = min(time_quantum, current->remaining_time);
        current->remaining_time -= execution_time;
        current_time += execution_time;

        int queue_size = ready_queue.size();
        for (int i = 0; i < queue_size; i++)
        {
            Process *p = ready_queue.front();
            ready_queue.pop();
            p->waiting_time += execution_time;
            ready_queue.push(p);
        }

        if (current->remaining_time > 0)
        {
            ready_queue.push(current);
        }
        else
        {
            completed++;
            current->turnaround_time = current_time;
        }
    }
}

```

```

    }

    void printResults(const vector<Process> &processes)
    {
        cout << "\nProcess\tBurst\tWaiting\tTurnaround\n";
        int total_waiting = 0;
        int total_turnaround = 0;

        for (const auto &p : processes)
        {
            cout << p.name << "\t" << p.burst_time << "\t"
                << p.waiting_time << "\t" << p.turnaround_time << endl;

            total_waiting += p.waiting_time;
            total_turnaround += p.turnaround_time;
        }

        float avg_waiting = static_cast<float>(total_waiting) / processes.size();
        float avg_turnaround = static_cast<float>(total_turnaround) / processes.size();

        cout << "\nAverage Waiting Time: " << avg_waiting << endl;
        cout << "Average Turnaround Time: " << avg_turnaround << endl;
    }

    int main()
    {
        int n, time_quantum;

        cin >> n >> time_quantum;

        vector<Process> processes(n);

        for (int i = 0; i < n; i++)
        {
            cin >> processes[i].name >> processes[i].burst_time;
        }

        roundRobinScheduling(processes, time_quantum);
        printResults(processes);

        return 0;
    }

```

4. Algorithm Trace

4.1 Input Parameters

- **Number of Processes:** 5
- **Time Quantum:** 2

- **Process Details:**

Process	Burst Time
P1	10
P2	1
P3	2
P4	1
P5	5

5. Results and Discussion

5.1 Process Execution Results

Process	Burst Time	Waiting Time	Turnaround Time
P1	10	9	19
P2	1	2	3
P3	2	3	5
P4	1	5	6
P5	5	10	15

5.2 Complete Execution Timeline

Time	Process	Action	Remaining Times
0-2	P1	Execute 2 units	P1:8, P2:1, P3:2, P4:1, P5:5
2-3	P2	Execute 1 unit (complete)	P1:8, P3:2, P4:1, P5:5
3-5	P3	Execute 2 units (complete)	P1:8, P4:1, P5:5
5-6	P4	Execute 1 unit (complete)	P1:8, P5:5
6-8	P5	Execute 2 units	P1:8, P5:3
8-10	P1	Execute 2 units	P1:6, P5:3
10-12	P5	Execute 2 units	P1:6, P5:1
12-14	P1	Execute 2 units	P1:4, P5:1
14-15	P5	Execute 1 unit (complete)	P1:4
15-17	P1	Execute 2 units	P1:2
17-19	P1	Execute 2 units (complete)	-

5.2 Gantt Chart

0	1	3	4	19
P2	P3	P4	P5	P1

6. Time and Space Complexity Analysis

6.1 Time Complexity

- **Best Case:** $O(n)$ - when all processes have burst time $\leq$ quantum
- **Average Case:** $O(n \times \text{avg_burst_time}/\text{quantum})$
- **Worst Case:** $O(n \times \text{max_burst_time})$ - when quantum = 1

7. Impact of Time Quantum

7.1 Small Time Quantum ($q \rightarrow 1$)

- **Advantages:** Better response time, more interactive
- **Disadvantages:** High context switching overhead, reduced throughput

7.2 Large Time Quantum ($q \rightarrow \infty$)

- **Advantages:** Reduced context switching, higher throughput
- **Disadvantages:** Approaches FCFS, poor response time

7.3 Optimal Quantum Selection

- Generally, 10-100ms in real systems

8. Conclusion

The Round Robin scheduling algorithm implementation successfully demonstrates key concepts of preemptive CPU scheduling. The experiment reveals important insights: It ensures fair allocation of CPU time to all processes in the ready queue. It also minimizes starvation by giving each process a fixed time slice repeatedly until completion.